

Universidade de São Paulo
Instituto de Ciências Matemáticas e de Computação
SCC-541 – Laboratório de Bases de Dados

Relatório – Trabalho Prático T4

Visões em PostgreSQL

Katiely Feitosa de Lacerda – 12777100
Leonardo Gonsalez – 15657074
Miguel Filippo Calhabeu – 15480331
Renan Silva Soriano – 11794824
Guilherme Motta Tranche – 13671549

São Carlos – 2026

1 Introdução

Este relatório apresenta o desenvolvimento e os testes realizados para o Trabalho Prático T4 da disciplina SCC-541 – Laboratório de Bases de Dados, sobre visões (views) em PostgreSQL. A base utilizada é a mesma construída ao longo da disciplina, composta por dados da Fórmula 1, cidades e aeroportos do mundo.

O objetivo principal foi explorar os diferentes tipos de visões oferecidos pelo PostgreSQL:

- Visões materializadas (MATERIALIZED VIEW) – armazenam fisicamente o resultado da consulta, com análise de desempenho e uso de REFRESH.
- Visões transientes (VIEW) – recalculadas a cada acesso, utilizadas para filtrar e combinar dados dinamicamente.
- Visões inline (WITH / CTE) – empregadas para isolar cálculos de distância e evitar repetição de código.

Foram desenvolvidas cinco visões principais:

- Aeroportos_Brasileiros – visão materializada com aeroportos de cidades brasileiras;
- Aeroportos_sem_cidades – aeroportos sem city_id associado;
- Cidades_brasileiras – cidades do Brasil com população $\geq 100\,000$ hab.;
- Problemas_aeroportos – aeroportos sem cidade a ≤ 10 km de cidades brasileiras;
- Correcao_aeroportos – visão atualizável usada para corrigir city_id.

Também foi criada a visão Circuitos_completa (Exercício 3), que consolida informações geográficas completas dos circuitos de Fórmula 1.

2 Exercício 1 – Visão Materializada Aeroportos_Brasileiros

2.1 Resolução

A visão materializada Aeroportos_Brasileiros foi criada a partir das tabelas airports, cities e countries, filtrando pelo código de país BR. Os atributos selecionados são nome e coordenadas do aeroporto, nome e continente do país e nome e população da cidade vinculada.

A escolha por visão materializada é justificada pelo volume dos dados: a tabela airports contém mais de 84 000 registros, e as junções com cities e countries são custosas. Armazenar o resultado pré-computado reduz significativamente o tempo de consulta.

```
CREATE MATERIALIZED VIEW Aeroportos_Brasileiros AS
  SELECT a.name, a.latitude_deg, a.longitude_deg,
         co.name AS pais_nome, cont.name AS continente, ci.name AS cidade_nome, ci.population
         AS cidade_populacao
  FROM airports a
  JOIN cities ci ON ci.id = a.city_id
  JOIN countries co ON co.id = ci.country_id
  WHERE co.code = 'BR';
```

2.2 Testes e Resultados

2.2.1 Exemplo de tuplas e contagem

A consulta `SELECT * FROM Aeroportos_Brasileiros LIMIT 10` retorna os primeiros aeroportos brasileiros cadastrados. A contagem total é obtida com `SELECT COUNT(*)`.

A visão contém 7 828 aeroportos brasileiros, incluindo pequenos aeródromos e heliportos.

aeroporto_nome	latitude_deg	longitude_deg	pais_nome	continente	cidade_nome	cidade_populacao
Escarpas Heliport	-16.212867	-48.509056	Brazil	South America	Abadiânia	NULL
Fazenda Mandarin Airport	-1.794319	-48.792022	Brazil	South America	Abaetetuba	158188

aeroporto_nome	latitude_deg	longitude_deg	pais_nome	continente	cidade_nome	cidade_populacao
JMG Agropecuária Airstrip	-19.023319	-45.411700	Brazil	South America	Abaeté	22675
Abaeté Airport	-19.155600	-45.494801	Brazil	South America	Abaeté	22675
Jeová Gomes Corrêa Airport	-8.736990	-39.115398	Brazil	South America	Abaré	17639
Fazenda Vale do Sol II Airstrip	-9.246739	-49.395683	Brazil	South America	Abreulândia	NU LL
Fazenda São Luiz Airport	-9.386899	-49.310819	Brazil	South America	Abreulândia	NU LL
Fazenda 4 Amigos Airport	-9.244835	-49.666942	Brazil	South America	Abreulândia	NU LL
Abunã Airstrip	-9.710450	-65.357930	Brazil	South America	Abunã	NU LL
Fazenda Santa Cármen Airstrip	-9.719693	-65.144742	Brazil	South America	Abunã	NU LL

2.2.2 Comparação de desempenho

Foram executadas pelo menos 5 consultas com EXPLAIN ANALYZE tanto na visão materializada quanto na consulta equivalente nas tabelas-base. Os resultados médios observados foram:

- Consulta direta nas tabelas-base: ~35,9 ms (3 Hash Joins + Seq Scans, EXPLAIN ANALYZE).
- Consulta na visão materializada: ~0,6 ms (Seq Scan direto na tabela materializada).
- Ganho de desempenho: fator de aproximadamente 62x (35,9 ms / 0,6 ms).
- Consumo de armazenamento da visão materializada: 1 032 kB (pg_total_relation_size).

A principal desvantagem é que a visão materializada não reflete alterações nas tabelas-base automaticamente — é necessário executar REFRESH MATERIALIZED VIEW.

2.2.3 Demonstração do REFRESH

Para demonstrar a necessidade do REFRESH, foi inserido um aeroporto fictício vinculado a uma cidade brasileira. A consulta à visão antes do REFRESH retorna 0 registros com o nome fictício; após o REFRESH, o registro aparece corretamente.

```
-- Antes do REFRESH: visão desatualizada
SELECT COUNT(*) FROM Aeroportos_Brasileiros WHERE aeroporto_nome LIKE '%T4%';
-- Resultado: 0

REFRESH MATERIALIZED VIEW Aeroportos_Brasileiros;

-- Após o REFRESH: visão atualizada
SELECT COUNT(*) FROM Aeroportos_Brasileiros WHERE aeroporto_nome LIKE '%T4%';
-- Resultado: 1
```

3 Exercício 2 – Aeroportos sem Cidade e Associação por Distância

3.1 Resolução

Foram criadas três visões para este exercício:

3.1.1 Aeroportos_sem_cidades

Filtra aeroportos com city_id IS NULL. Por definição, esses aeroportos não podem ser classificados por país a partir da modelagem atual.

```
CREATE VIEW Aeroportos_sem_cidades AS
SELECT id, name, latitude_deg, longitude_deg
FROM airports
WHERE city_id IS NULL;
```

3.1.2 Cidades_brasileiras

Seleciona cidades do Brasil com população de no mínimo 100 000 habitantes, incluindo nome, população e coordenadas.

```
CREATE VIEW Cidades_brasileiras AS
  SELECT ci.id, ci.name, ci.population, ci.latitude, ci.longitude
  FROM cities ci
  JOIN countries co ON co.id = ci.country_id
  WHERE co.code = 'BR' AND ci.population >= 100000;
```

3.1.3 Associação por distância ≤ 10 km

A função Earth_Distance (extensão EarthDistance + CUBE) calcula a distância em metros entre dois pontos. Usamos uma visão inline (WITH) para isolar o cálculo e evitar repetição:

```
WITH distancias AS (
  SELECT a.name AS aeroporto_nome, c.name AS cidade_nome,
         c.population,
         earth_distance(
           ll_to_earth(a.latitude_deg, a.longitude_deg),
           ll_to_earth(c.latitude, c.longitude)
         ) / 1000.0 AS distancia_km
  FROM Aeroportos_sem_cidades a
  CROSS JOIN Cidades_brasileiras c
)
SELECT * FROM distancias WHERE distancia_km <= 10
ORDER BY aeroporto_nome, distancia_km;
```

3.2 Testes

A consulta retorna os pares (aeroporto sem cidade, cidade brasileira) cuja distância é inferior ou igual a 10 km, apresentando nome do aeroporto, nome da cidade, população e distância arredondada em km.

A consulta retornou 1 aeroporto: "Guarulhos - Sao Paulo Airport Marriott Heliport" (id 275301), com 9 cidades brasileiras candidatas a ≤ 10 km. A mais próxima é Guarulhos (1 169 577 hab., 4,313 km).

4 Exercício 3 – Visão Circuitos_completa

4.1 Resolução

A visão Circuitos_completa consolida as informações de localização geográfica de todos os circuitos de Fórmula 1 cadastrados, independentemente de estarem vinculados a uma cidade. Para isso, foram utilizados LEFT JOINS, garantindo que circuitos sem city_id também apareçam no resultado.

```
CREATE VIEW Circuitos_completa AS
  SELECT ci.name AS circuito_nome, ci.lat, ci.long,
         ct.name AS cidade_nome,
         co.name AS pais_nome, co.code AS pais_code,
         cont.name AS continente_nome
  FROM circuits ci
  LEFT JOIN cities ct ON ct.id = ci.city_id
  LEFT JOIN countries co ON co.id = ct.country_id
  LEFT JOIN continents cont ON cont.id = co.continent_id;
```

A escolha por LEFT JOIN em vez de INNER JOIN é justificada pelo enunciado: "deve apresentar todos os circuitos, independentemente de estarem ou não vinculados a alguma cidade".

4.2 Testes

Número de tuplas: a visão contém 76 tuplas (um circuito por linha), correspondendo a todos os circuitos cadastrados na tabela circuits.

A consulta `SELECT * FROM Circuitos_completa LIMIT 10` retorna exemplos variados incluindo circuitos europeus (Silverstone, Mônaco, Monza), americanos (Interlagos, Circuit of the Americas) e asiáticos (Suzuka, Marina Bay).

Circuitos sem cidade vinculada (`city_id NULL`) aparecem com os campos `cidade_nome`, `pais_nome` e `continente_nome` preenchidos como `NULL`, demonstrando que o `LEFT JOIN` foi aplicado corretamente.

5 Exercício 4 – Visão Problemas_aeroportos

5.1 Resolução

A visão `Problemas_aeroportos` filtra, a partir de `Aeroportos_sem_cidades`, apenas aqueles que possuem ao menos uma cidade da visão `Cidades_brasileiras` a ≤ 10 km de distância. O cálculo de distância é encapsulado em uma CTE para maior clareza:

```
CREATE VIEW Problemas_aeroportos AS
WITH distancias AS (
    SELECT a.id, a.name, a.latitude_deg, a.longitude_deg,
           c.name AS cidade_candidata, c.population,
           earth_distance(
               ll_to_earth(a.latitude_deg, a.longitude_deg),
               ll_to_earth(c.latitude, c.longitude)
           ) / 1000.0 AS distancia_km
    FROM Aeroportos_sem_cidades a
    CROSS JOIN Cidades_brasileiras c
)
SELECT id, name, latitude_deg, longitude_deg,
       cidade_candidata, population AS populacao,
       ROUND(distancia_km::numeric, 3) AS distancia_km
FROM distancias
WHERE distancia_km <= 10;
```

5.2 Testes e Explicação

(b) Por que esses aeroportos aparecem nessa visão? Os aeroportos listados possuem `city_id NULL` na tabela `airports` — ou seja, a base não registra uma cidade vinculada a eles. Entretanto, suas coordenadas geográficas (latitude e longitude) estão a menos de 10 km de cidades brasileiras com população de ao menos 100 000 habitantes. Esse cenário indica prováveis inconsistências de carga de dados: o aeroporto existe fisicamente próximo a uma grande cidade, mas a associação não foi inserida corretamente na base.

Para cada tupla retornada, o campo `cidade_candidata` e a distância em km indicam qual cidade brasileira é a mais provável candidata a ser vinculada ao aeroporto.

6 Exercício 5 – Visão Correcao_aeroportos e Atualização

6.1 Resolução

A visão `Correcao_aeroportos` seleciona diretamente de `airports` os atributos `id`, `name`, `latitude_deg`, `longitude_deg` e `city_id`, sem nenhuma junção, filtrada pelos aeroportos identificados em `Problemas_aeroportos`. Por derivar de uma única tabela sem agregações ou operações de conjunto, ela é automaticamente atualizável (AUVis) no PostgreSQL.

```
CREATE VIEW Correcao_aeroportos AS
SELECT id, name, latitude_deg, longitude_deg, city_id
FROM airports
WHERE id IN (SELECT DISTINCT id FROM Problemas_aeroportos);
```

6.2 Identificação de Cidades Candidatas e UPDATE

Para cada aeroporto presente em `Correcao_aeroportos`, foi selecionada a cidade mais próxima (menor distância) dentre as cidades da visão `Cidades_brasileiras`. O UPDATE é realizado diretamente na visão:

```
WITH cidade_mais_proxima AS (
    SELECT DISTINCT ON (id)
        id AS aeroporto_id,
        (SELECT c.id FROM cities c
         JOIN countries co ON co.id = c.country_id
         WHERE co.code = 'BR' AND c.name = p.cidade_candidata
         LIMIT 1) AS cidade_id
    FROM Problemas_aeroportos p
    ORDER BY id, distancia_km
)
UPDATE Correcao_aeroportos ca
SET city_id = cmp.cidade_id
FROM cidade_mais_proxima cmp
WHERE ca.id = cmp.aeroporto_id
AND cmp.cidade_id IS NOT NULL;
```

6.3 Verificação dos Efeitos

Após o UPDATE, as visões `Aeroportos_sem_cidades` e `Problemas_aeroportos` são consultadas novamente para verificar que os aeroportos corrigidos não aparecem mais:

```
-- Deve retornar 0 para os aeroportos corrigidos
SELECT COUNT(*) FROM Aeroportos_sem_cidades
WHERE id IN (SELECT id FROM Correcao_aeroportos);

-- Confirma city_id atualizado
SELECT ca.id, ca.name, ca.city_id, ci.name AS cidade_vinculada
FROM Correcao_aeroportos ca
JOIN cities ci ON ci.id = ca.city_id
ORDER BY ca.name LIMIT 20;
```

Resultado real: 1 aeroporto corrigido — id 275301 ("Guarulhos - Sao Paulo Airport Marriott Heliport") recebeu city_id = 3461786 (Guarulhos). Após o UPDATE, Problemas_aeroportos retorna 0 linhas.

7 Conclusão

Este trabalho demonstrou na prática as diferenças e aplicabilidades dos tipos de visões disponíveis no PostgreSQL:

- reduziu o tempo de consulta em ~62x (de ~35,9 ms para ~0,6 ms) ao pré-computar as junções, mas exige REFRESH manual para refletir alterações nas tabelas-base.
- Visões transientes (`Aeroportos_sem_cidades`, `Cidades_brasileiras`, `Circuitos_completa`, `Problemas_aeroportos`): sempre refletem o estado atual das tabelas-base, mas têm custo de execução a cada acesso.
- Visão atualizável (`Correcao_aeroportos`): permitiu corrigir o atributo `city_id` diretamente via UPDATE na visão, aproveitando a propriedade AUVIS do PostgreSQL para visões derivadas de uma única tabela.
- Visões inline (WITH/CTE): utilizadas para isolar o cálculo de distância Haversine via `Earth_Distance`, melhorando a legibilidade e evitando repetição de código.

As decisões adotadas buscaram manter as soluções aderentes ao esquema real da base, evitar alterações desnecessárias na estrutura e centralizar a lógica nos próprios objetos do SGBD.